The AKS Algorithm

Siddharth & Amit

March 24, 2026

The Breakthrough

The Question

Is primality testing in **P**?

- Fermat: fast but unreliable
- Miller-Rabin: fast, probabilistic

AKS (2002)

PRIMES is in P

What AKS Achieves

- Deterministic
- Unconditional
- Polynomial time

$$\text{Time: } \mathcal{O}((\log n)^c)$$

Significance
First proof that primality testing is efficiently solvable

Key Identity

The Core Idea

Key Identity
n is prime iff:

$$(X + a)^n \equiv X^n + a \pmod{n}$$

- Holds for all a with $\gcd(a, n) = 1$
- Comes from binomial expansion

Proof ($\Rightarrow$ direction)

Assume n is prime.

$$(X + a)^n = \sum_{i=0}^{n} \binom{n}{i} a^{n-i} X^i$$

For $0 < i < n$:

$$\binom{n}{i} = \frac{n!}{i!(n-i)!}$$

$$\Rightarrow \binom{n}{i} \equiv 0 \pmod{n}$$

Proof (continued)

Thus:

$$(X + a)^n \equiv X^n + a^n \pmod{n}$$

By Fermat's Little Theorem:

$$a^n \equiv a \pmod{n}$$

$$\Rightarrow (X + a)^n \equiv X^n + a \pmod{n}$$

Proof ($\Leftarrow$ direction)

Assume:

$$(X + a)^n \equiv X^n + a \pmod{n}$$

Suppose n is composite.

Let $q^k \parallel n$ (largest power of q dividing n).

Proof (continued)

Consider coefficient:

$$\binom{n}{q} = \frac{n(n-1)\cdots(n-q+1)}{q!}$$

$$\binom{n}{q} = \frac{q^{k-1}(n-1)\cdots(n-q+1)}{(q-1)!}$$

$$\Rightarrow \binom{n}{q} \not\equiv 0 \pmod{n}$$

Example: Prime Case ($n = 5$)

$$(X+1)^5 = X^5 + 5X^4 + 10X^3 + 10X^2 + 5X + 1$$

Modulo 5:

$$(X+1)^5 \equiv X^5 + 1 \pmod 5$$

Observation
All middle coefficients vanish modulo 5

Pascal Triangle Insight

Row 5 of Pascal's Triangle:

$$
\begin{array}{c}
1 \\
1\ 1 \\
1\ 2\ 1 \\
1\ 3\ 3\ 1 \\
1\ 4\ 6\ 4\ 1 \\
1\ 5\ 10\ 10\ 5\ 1
\end{array}
$$

- For prime p, all interior entries are divisible by p
- Only the endpoints are 1

$$\Rightarrow (X + a)^p \equiv X^p + a \pmod{p}$$

Example: Composite Case ($n = 4$)

$$(X+1)^4 = X^4 + 4X^3 + 6X^2 + 4X + 1$$

Modulo 4:

$$(X+1)^4 \equiv X^4 + 2X^2 + 1 \pmod 4$$

Observation
Middle coefficient $6 \equiv 2 \pmod 4$ does not vanish

$$\Rightarrow \text{Identity fails}$$

Why Not Use This Directly?

- The identity gives a **perfect characterization** of primes

$$(X + a)^n = \sum_{i=0}^{n} \binom{n}{i} a^{n-i} X^i$$

- Polynomial has $n + 1$ terms
- Computing it requires $\Omega(n)$ operations

Problem
Input size is $\log n$, but runtime is $\Omega(n)$

Conclusion
Exponential time $\Rightarrow$ not efficient

The Trick

Reducing the Cost

Work modulo:

$$X^r - 1$$

- $X^r \equiv 1$
- Degrees wrap around

Effect
Polynomial size reduces from n to r

New Test

Check:

$$(X + a)^n \equiv X^n + a \pmod{X^r - 1,\ n}$$

- Much faster
- Works for primes

Issue
Some composites may now pass

Fixing False Positives

- Work modulo $X^r - 1$ (compression)
- Test multiple values of a

Key Condition

$$\mathrm{ord}_r(n) > (\log n)^2$$

- Ensures composite numbers cannot satisfy identity for many a

What is $\mathrm{ord}_r(n)$?

Definition

$$\mathrm{ord}_r(n) = \min\{k > 0 \mid n^k \equiv 1 \pmod{r}\}$$

- Measures how n behaves modulo r
- Captures the **cycle length** of powers of n

Example

$$2^3 \equiv 1 \pmod{7} \Rightarrow \mathrm{ord}_7(2) = 3$$

Why Do We Need Large Order?

- We want:

$$\mathrm{ord}_r(n) > (\log n)^2$$

Reason
Ensures n does not behave "periodically" too early modulo r

- Prevents composite numbers from mimicking primes
- Forces large algebraic structure in the proof

Existence of r

Lemma
There exists $r \leq (\log n)^5$ such that:

$$\mathrm{ord}_r(n) > (\log n)^2$$

Proof.

Let:

$$B = \lceil (\log n)^5 \rceil$$

Assume for contradiction that for all $r \leq B$:

$$\mathrm{ord}_r(n) \leq (\log n)^2$$

Key Construction

Then for every $r \leq B$, there exists $k \leq (\log n)^2$ such that:

$$n^k \equiv 1 \pmod{r}$$

$$\Rightarrow r \mid (n^k - 1)$$

Thus every $r \leq B$ divides:

$$N = \prod_{k=1}^{\lfloor (\log n)^2 \rfloor} (n^k - 1)$$

Bounding N

Each term:

$$n^k - 1 < n^k$$

So:

$$N < \prod_{k=1}^{(\log n)^2} n^k = n^{\sum_{k=1}^{(\log n)^2} k}$$

$$= n^{\frac{(\log n)^2((\log n)^2+1)}{2}} \leq n^{(\log n)^4}$$

$$= 2^{(\log n)^5}$$

Contradiction via LCM

But if every $r \leq B$ divides N, then:

$$\mathrm{lcm}(1, 2, \ldots, B) \mid N$$

Fact

$$\mathrm{lcm}(1, 2, \ldots, B) \geq 2^B$$

Thus:

$$2^B \leq N < 2^{(\log n)^5}$$

Final Contradiction

Recall:

$$B = (\log n)^5$$

So:

$$2^B = 2^{(\log n)^5}$$

But we showed:

$$N < 2^{(\log n)^5}$$

$$\Rightarrow 2^B \nmid N$$

Contradiction
Not all $r \leq B$ can divide N

Conclusion

- There exists some $r \leq (\log n)^5$ such that:

$$\mathrm{ord}_r(n) > (\log n)^2$$

Key Insight
If all orders were small, too many numbers would divide too small a number

The Algorithm

AKS Algorithm

1. Check if $n = a^b$ for some $b > 1$
2. Find smallest r with $\mathrm{ord}_r(n) > (\log n)^2$
3. If $\gcd(a, n) > 1$ for some $a \leq r$, output composite
4. If $n \leq r$, output prime
5. For $a = 1$ to $O(\sqrt{r} \log n)$:

$$(X + a)^n \equiv X^n + a \pmod{X^r - 1, n}$$

6. Output prime

Why Each Step Matters

- Step 1: removes perfect powers
- Step 2: ensures strong algebraic structure
- Step 3: detects small factors quickly
- Step 4: handles small cases
- Step 5: main polynomial test

Key Point
All steps together eliminate every possible composite

Why the Polynomial Test Works

- Identity must hold for primes
- Modulo $X^r - 1$, information is compressed

Key Idea
If identity holds for many a, then n behaves like a prime in a strong algebraic sense

Correctness: Setup

Assume the algorithm reaches the final step and outputs $\textsc{Prime}$.

Then:

- n is not a perfect power
- $\gcd(a, n) = 1$ for all $a \leq r$
- $\mathrm{ord}_r(n) > (\log n)^2$
- For all $1 \leq a \leq \ell$:

$$(X + a)^n \equiv X^n + a \pmod{X^r - 1,\ n}$$

Let p be any prime divisor of n.

Reducing Modulo p

Since $p \mid n$:

$$(X + a)^n \equiv X^n + a \pmod{X^r - 1,\ p}$$

Also, since p is prime:

$$(X + a)^p \equiv X^p + a \pmod{p}$$

Raise both sides to power n/p:

$$(X + a)^n \equiv (X^p + a)^{n/p}$$

Key Identity

Combining:

$$(X^p + a)^{n/p} \equiv X^n + a \pmod{X^r - 1,\ p}$$

Let $Y = X^p$:

$$(Y + a)^{n/p} \equiv Y^{n/p} + a$$

Observation
n/p satisfies the same identity as a prime

Introspective Property

Define:

$$f(X)^m \equiv f(X^m) \pmod{X^r - 1,\ p}$$

Then:

- p satisfies this for $f(X) = X + a$
- n/p also satisfies it

Closure
Products of such numbers also satisfy this property

Building a Large Set

Define:

$$I = \{(n/p)^i \cdot p^j \mid i, j \geq 0\}$$

All elements of I satisfy:

$$f(X)^m \equiv f(X^m)$$

Reduce modulo r:

$$G = \{m \bmod r \mid m \in I\}$$

$$|G| > (\log n)^2$$

Another Structure

Consider polynomials:

$$P = \left\{ \prod (X + a)^{e_a} \right\}$$

Their images modulo an irreducible factor form a group:

$$\hat{G}$$

Key Result

$$|\hat{G}| > n^{\sqrt{|G|}}$$

Upper Bound

If n is not a power of p:

$$|\hat{G}| \leq n^{\sqrt{|G|}}$$

Contradiction

$$|\hat{G}| > n^{\sqrt{|G|}} \quad \text{and} \quad |\hat{G}| \leq n^{\sqrt{|G|}}$$

Conclusion

Thus:

$$n = p^k$$

From Step 1:

$$k = 1$$

Final Result

$$n = p \Rightarrow n \text{ is prime}$$

Complexity Overview

Let input size be:

$$k = \log n$$

Goal
Show AKS runs in polynomial time in k

We analyze each step of the algorithm.

Step 1: Perfect Power Check

- Try all exponents $b \leq \log n$
- Compute integer b-th roots

$$\text{Time} = \mathcal{O}((\log n)^3)$$

Reason
Polynomial-time arithmetic on $\log n$-bit numbers

Step 2: Finding r

We search for smallest r such that:

$$\mathrm{ord}_r(n) > (\log n)^2$$

- Try $r \leq (\log n)^5$
- For each r, test powers up to $(\log n)^2$

$$\text{Time} = \mathcal{O}((\log n)^7)$$

Step 3: GCD Checks

- Compute $\gcd(a, n)$ for $a \leq r$

$$r = \mathcal{O}((\log n)^5)$$

$$\text{Time per gcd} = \mathcal{O}(\log n)$$

$$\Rightarrow \text{Total} = \mathcal{O}((\log n)^6)$$

Step 5: Main Bottleneck

We check:

$$(X + a)^n \equiv X^n + a \pmod{X^r - 1,\ n}$$

- Number of a values:

$$\ell = \mathcal{O}(\sqrt{r} \log n)$$

$$r = \mathcal{O}((\log n)^5) \Rightarrow \ell = \mathcal{O}((\log n)^{7/2})$$

Cost Per Iteration

Compute $(X + a)^n \bmod (X^r - 1, n)$:

- Use repeated squaring: $\mathcal{O}(\log n)$ steps
- Each multiplication:
  - Degree $\leq r$
  - Coefficients size $\log n$

$$\text{Cost per multiplication} = \mathcal{O}(r \log n)$$

$$\Rightarrow \text{Per iteration} = \mathcal{O}(r(\log n)^2)$$

Total Complexity

$$\text{Total} = \ell \cdot \mathcal{O}(r(\log n)^2)$$

Substitute:

$$\ell = (\log n)^{7/2}, \quad r = (\log n)^5$$

$$\Rightarrow \mathcal{O}((\log n)^{7/2} \cdot (\log n)^5 \cdot (\log n)^2)$$

$$= \mathcal{O}((\log n)^{21/2})$$

Improvements

- Better bounds on r reduce complexity
- Improved algorithms achieve:

$$\mathcal{O}((\log n)^6)$$

Key Insight
All steps are polynomial in $\log n$

Comparison

- Fermat: fast, unreliable
- Miller–Rabin: fast, probabilistic
- AKS: deterministic, polynomial

Trade-off
AKS is theoretically important, but slower in practice

Takeaway

Randomness is not necessary.

Conclusion
Primality testing is fundamentally efficient